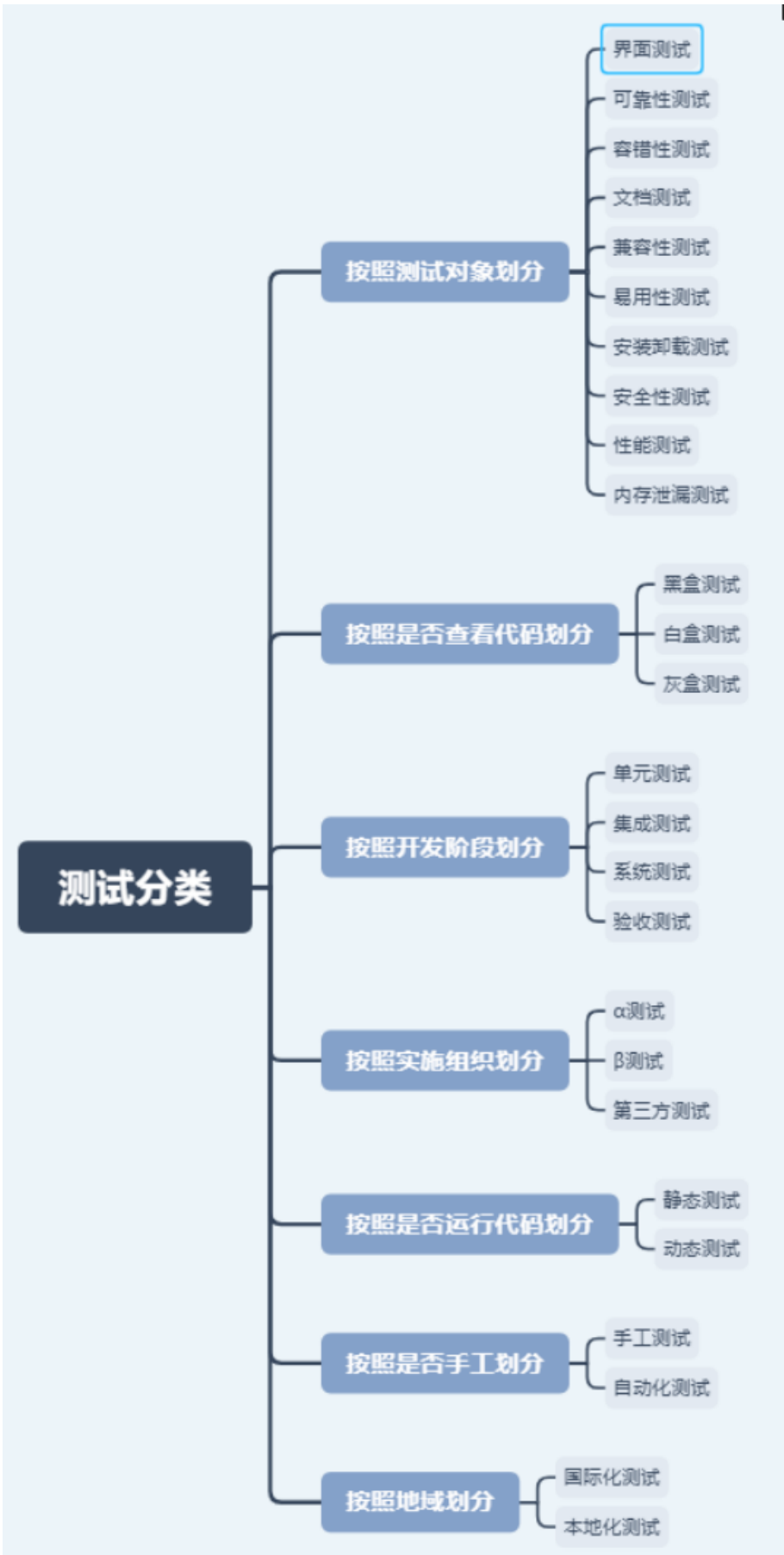


第三章 软件测试方法

- 概览



- 黑盒测试

黑盒测试(Black-box Testing)

黑盒测试就是在完全不考虑程序逻辑和内部结构的情况下，检查系统功能是否按照需求规格说明书的规定正常使用、是否能适当的接收输入数据而输出正确的结果，满足规范需求。

所以，黑盒测试又称之为数据驱动测试，只注重软件的功能

黑盒测试的优点

- 不需要了解程序内部的代码以及实现，不关注软件内部的实现。
- 从用户角度出发设计测试用例，很容易的知道用户会用到哪些功能，会遇到哪些问题，锻炼测试人员的产品思维
- 测试用例是基于软件需求开发文档，不容易遗漏软件需求文档中需要测试的功能。

黑盒测试的缺点是不可能覆盖所有代码。

黑盒测试用到的测试方法有，等价类，边界值，因果图，场景法，错误猜测法等。

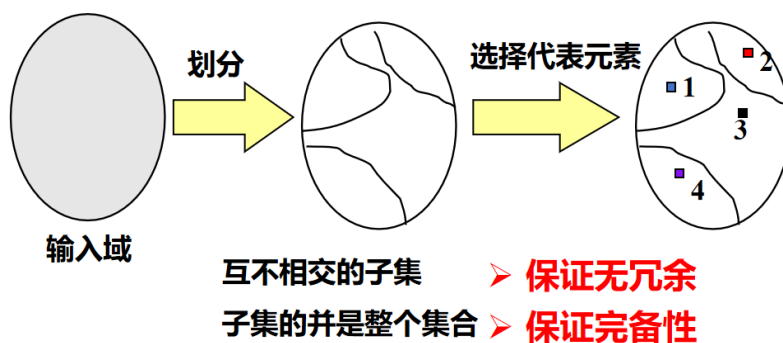
- 3.2 基于输入域的方法

- 基于输入域的方法包括边界值分析法和等价类划分法。
- 关于输入域：
 - 整体输入域：由多个输入条件共同构成的具有一定实际意义的输入域。
 - 个体输入域：由各个输入条件分别构成的单个输入域的集合。

- 3.2.1 等价类划分法

- 1、基本思想

- 把程序的输入域划分成若干部分，然后从每个部分中选取少数代表性数据当作测试用例，每一类的代表性数据在测试中的作用等价于这一类中的其它值。（保证全覆盖）
 - 划分等价类的原则和目标：完备，无冗余。



- 基于：独立性假设+单缺陷假设。
 - 等价类测试适用于：可用数量衡量的独立变量、布尔变量、逻辑变量；不适用于：相互有依赖关系的变量
- 2、等价类测试的步骤
 - 1.划分等价类
 - 有效等价类和无效等价类

- 有效等价类：对于程序的规格说明而言，是合理的、有意义的输入数据构成的集合（考察正常工作能力）
- 无效等价类：对于程序的规格说明而言，是不合理的、无意义的输入数据构成的集合（考察软件的容错能力）

• 2.列出等价类表

输入条件	有效等价类	编号	无效等价类	编号
		(1)		(7)
		(2)		(8)
		(3)		(9)
		(4)		(10)
				(11)
		(5)		(12)
		(6)		(13)

• 3.设计测试用例

1. 为每个等价类规定一个唯一的编号；
2. 设计一个新的测试用例，使之尽可能多地覆盖尚未被覆盖的有效等价类，重复这一步直至所有的有效等价类都被覆盖为止；
3. 设计一个新的测试用例，使其仅覆盖一个尚未被覆盖的无效等价类，重复这一步直至所有的无效等价类都被覆盖为止。

用例序号	输入参数 (输入的数据)	覆盖的等价类编号	输出参数
1			
2			
3			
4			
5			
6			

• 3、示例

- 余额宝提现的等价类划分
 - 余额宝中的零钱不可以直接消费，当需要使用时，可将余额宝的钱提现。余额宝的提现功能有2种方式：
 - 快速到账（2小时内）：每日最高提现额度为10000元；
 - 普通到账：可提取金额为余额宝最大余额，但到账时间会慢一些。

□ 选择快速到账，则可将提现功能划分为1个有效等价类与2个无效等价类，具体如下：

- 1、有效等价类：0<提现金额<=10000元。
- 2、无效等价类：提现金额<=0。
- 3、无效等价类：提现金额>10000元。

□ 选择普通到账，则可将提现功能划分为1个有效等价类和2个无效等价类，具体如下：

- 1、有效等价类：0<提现金额<=余额。
- 2、无效等价类：提现金额<=0。
- 3、无效等价类：提现金额>余额。

● 划分1

功能	有效等价类	编号	无效等价类	编号
快速到账	0<提现金额<=10000元	1	提现金额<=0	2
			提现金额>10000元	3
普通到账	0<提现金额<=余额	4	提现金额<=0	5
			提现金额>余额	6

● 更正后的划分2

功能	有效等价类	编号	无效等价类	编号
快速到账 (第1次)	0<提现金额<=10000元	1	提现金额<=0	2
			提现金额>10000元	3
快速到账 (第n次)	0<提现金额<=10000元-已提现金额	7	提现金额<=0	8
			提现金额>10000元-已提现金额	9
普通到账	0<提现金额<=余额	4	无效等价类：提现金额<=0	5
			提现金额>余额	6

● 测试用例

假如现在余额宝中有50000元
覆盖有效等价类

测试用例	功能	金额（元）	覆盖有效等价类编号
Test1	快速到账（第1次）	1000	1
Test2	快速到账（第n次，已提现2000元）	7000	7
Test3	普通到账	40000	4

假如现在余额宝中有50000元
覆盖无效等价类

测试用例	功能	金额 (元)	覆盖有效等价类编号
Test4	快速到账 (第1次)	-1000	2
Test5		20000	3
Test6	快速到账 (第n次, 已提现2000元)	-2000	8
Test7		9000	9
Test8	普通到账	-3000	5
Test9		60000	6

4、等价类测试用例的设计

1.一般/健壮, 强/弱等价类的定义

• **基于假设:** 输入变量是真正独立的;

• **分类的决定因素:**

• **<一般 (有效), 健壮 (无效) >**

• **<弱 (单缺陷), 强 (多缺陷) >**

• **方法分类:** 弱一般等价类测试、强一般等价类测试、弱健壮等价类测试、强健壮等价类测试;

• **可用于程序的输入定义域或输出值域**

• “单缺陷”假设: 假定被测试的软件只存在一个缺陷或错误。

• “多缺陷”假设: 假定被测试的软件可能存在多个缺陷或错误。

2.方法分类

• **弱一般等价类测试**

• **强一般等价类测试**

• **弱健壮等价类测试**

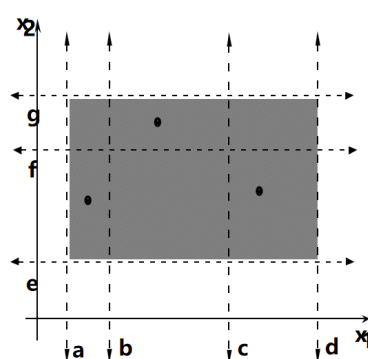
• **强健壮等价类测试**

• 单缺陷与多缺陷假设产生**弱等价类**与**强等价类**测试之分。

• 是否进行无效数据的处理产生**健壮**与**一般**等价类测试之分。

弱一般等价类

• **基于单缺陷、正常值**



有效等价类	
对于 x1	M1={x1 a<=x1<b}
	M2={x1 b<=x1<c}
	M3={x1 c<=x1<=d}
对于 x2	N1={x2 e<=x2<f}
	N2={x2 f<=x2<=g}

测试用例	输入等价类	输入等价类
1	M1,N1	M1,N2
2	M2,N2	M2,N1
3	M3,N1	M3,N2

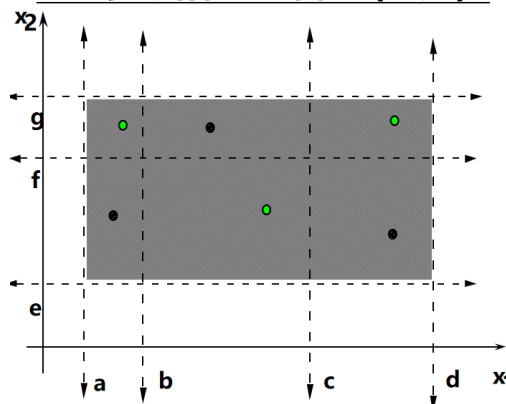
对于n个变量, 设第i个变量的有效等价类是 m_i 个, 则测试用例总数:
 $\max(m_i)$ 个。

• 对有效输入, 使每个有效等价类有一个值即可。

- 不考虑无效输入。

- 强一般等价类

- 基于多缺陷、正常值（一般）



测试用例	输入等价类
1	M1,N1
2	M1,N2
3	M2,N1
4	M2,N2
5	M3,N1
6	M3,N2

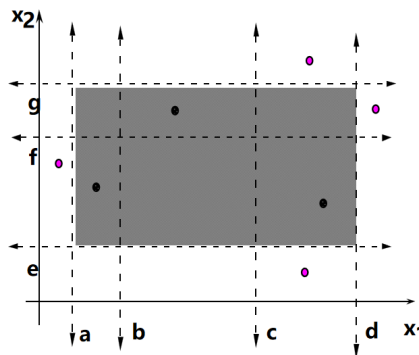
对于n变量，设第i个变量的有效等价类是 m_i 个，则测试用例总数：

$$m_1 * m_2 * \dots * m_n$$

基于多缺陷假设需要等价类笛卡尔积的每个元素对应的测试用例

- 弱健壮等价类

- 基于单缺陷、健壮值



	无效等价类
对于x1	M4={x1 x1<a} M5={x1 x1>d}
对于x2	N3={x2 x2<e} N4={x2 x2>g}

测试用例	补充输入等价类
1,2	(M4,N1)(M4,N2)
3,4	(M5,N1)(M5,N2)

对于n变量，设第i个变量的有效等价类是 m_i 个，无效等价类是 l_i 个，则测试用例总数：

$$\max(m_i) + (l_1 + l_2 + \dots + l_n) \text{ 个。}$$

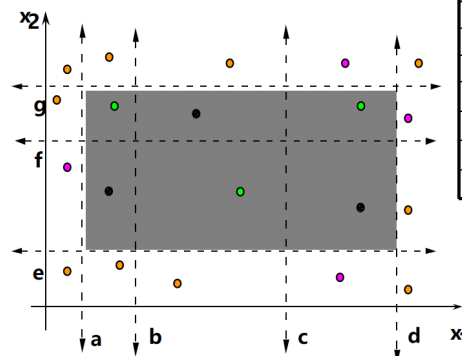
- 弱--基于单缺陷假设，健壮--考虑无效值

- 具体做法：

- 对有效输入，使用每个有效等价类的一个值。
- 对无效输入，测试用例将拥有一个无效值并保持其余的值都是有效的。

- 强健壮等价类

- 基于多缺陷、健壮值



测试用例	输入等价类
1,2	(M1,N1)(M1,N2)
3,4	(M2,N1)(M2,N2)
5,6	(M3,N1)(M3,N2)
7,8	(M4,N1)(M4,N2)
9,10	(M5,N1)(M5,N2)
11,12	(M1,N3)(M1,N4)
...	...

对于n变量，设第i个变量的有效等价类是 m_i 个，无效等价类是 l_i 个，则测试用例总数：

$$(m_1 + l_1) * (m_2 + l_2) * \dots * (m_n + l_n)$$

- 3.总结

- 弱一般等价类：单缺陷，正常值，测试用例总数= $\max\{m1, m2\}$
- 强一般等价类：多缺陷，正常值，测试用例总数= $m1*m2$
- 弱健壮等价类：单缺陷，健壮值，测试用例总数= $\max\{m1, m2\}$ +无效等价类个数
- 强健壮等价类：多缺陷，健壮值，测试用例总数= $(m1+l1)*(m2+l2)*\dots*(mn+ln)$
- 其中 m_i 指的是第 i 个变量的有效等价类是 m 个， l_i 指的是第 i 个变量的无效等价类是 l 个。

4.示例-第二日

- 例二：NextDate 输入定义域测试

输入条件的有效等价类				输入条件的无效等价类			
M1={月份 1<=月<=12}				M2={月份 月<1}			
D1={日期 1<=日期<=31}				M3={月份 月>12}			
Y1={年 1800<=年<=2050}				D2={日期 日<1}			
				D3={日期 日>31}			
				Y2={年 年<1800}			
				Y3={年 年>2050}			
测试用例总数: (个)							
弱一般	强一般	弱健壮	强健壮				
1	1	7	27				

- 弱一般：单缺陷，正常值，实际上如果值正常了就不能说存在缺陷， $\max\{1,1,1\}=1$
- 强一般：强缺陷，正常值， $1*1*1=1$
- 弱健壮：只是某个无效，其他无效，考虑无效值， $\max\{1,1,1\}$ +无效等价类个数=7
- 强健壮：无效的全排列，考虑无效值， $(1+2)^3=27$

5、习题

- 雨课堂作业3-1

等价类测试

课后练习1:

- 根据“职工信息登记系统”的数据要求，出每个填写项的有效等价无效等价类界值，并编写测试用例。
- 数据要求: “
 - 编号:1--5000之间的整数，不能为空，不能重复;
 - 姓名:3--10个字符，不能为空;
 - 工资:1.00-30000.00 之间的小数(或整数)，可以为空，小数点后最多保留2位;
 - 住址:0--30个字符。

实验2附录3

3、验证电话号码

- 要求：城市的电话号码由两部分组成。这两部分的名称和内容分别是：
 - 地区码：以 0 开头的三位或者四位数字（包括 0）；
 - 电话号码：以非 0、非 1 开头的七位或者八位数字。
 假定被调试的程序能接受一切符合上述规定的电话号码，拒绝所有不符合规定的号码。
 请使用等价类划分法来设计测试用例。

- 实验2附录4

- 4、验证准考证号码

- 要求：对招干考试系统“输入学生成绩”子模块设计测试用例。

- 招干考试分三个专业，准考证号第一位为专业代号，如：

- 1-行政专业，

- 2-法律专业，

- 3-财经专业。

- 行政专业准考证号码为：110001~111215

- 法律专业准考证号码为：210001~212006

- 财经专业准考证号码为：310001~314015

- 请划分准考证号码的等价类。

- PPT习题

- 信息注册：

- 姓名：1-20个字符，不能包含数字，不能为空；
 - 年龄：18-60之间的整数，不能为空；
 - 如果填写信息正确，给出提示信息，如图：
 - 并在“注册信息”文本框中输入相应注册信息。



- 3.2.2 边界值分析法

- 1、原理

- 在被测对象的边界及边界附近设计测试用例。
 - 是一种最基本、最简单的黑盒测试方法，通常可作为等价类测试的补充。
 - 基于独立性假设和单缺陷假设。

- 2、测试步骤

- 1.选择被测对象。

- 整体输入域（多个输入条件、边界清晰但难以覆盖）。
 - 个体输入域（单个输入条件、边界不清晰）。

- 2.确定边界。

- 边界的处理技巧（课本P47）
 - 值的范围：取刚刚达到这个范围的边界值和刚刚超过这个范围边界的值。
 - 值的个数：最大/最小，比最大多一/比最小少一。
 - 有序集合/顺序表：取第一个和最后一个元素。

- 光标：最左上，最右下。
- n比特整数： $2^{(n-1)}$ 和 $-2^{(n-1)}$ 是边界。
- 循环的第0次，1次，倒数第二次，倒数第一次。
- **3.确定邻域（即输入/输出域边界附近的邻域范围）**
 - 邻域的选择
 - 对于每个输入条件的每个边界点，需在该点附近确定大小为1的邻域。测试用例是基于所有输入条件的所有边界点及其邻域来设计的。（这里的“1”是指1个单位长度，并非数字意义上的“1”。）
 - 上点、内点、离点的概念
 - 上点：边界上的点。
 - 内点：范围内的点。
 - 离点：离边界点最近的点，开内闭外（开区间，离点在范围内；闭区间，离点在范围外）。
- **4.设计用例（即从边界及其邻域抽取测试数据，设计测试用例）**
 - (1)测试数据的选择
 - 穷举法：在每个边界点的邻域范围内取所有数据——>所有数据均可测试，负担重
 - 典型值法：a处选择a-1,a,a+1
 - (2)边界组合方式的选择
 - 强边界法：测试用例覆盖所有输入条件的所有边界组合。可测试到所有的边界组合，但不利于缺陷的隔离和定位。
 - 弱边界法：单缺陷假设，优势在于便于快速隔离和定位边界缺陷，且大大降低测试用例。
 - 全边界法：强边界+弱边界
- **雨课堂习题3-3**

练习2：126邮箱注册功能——边界值法

测试项	测试点	需求规格	有效等价类	测试数据	编号	无效等价类	测试数据	编号
用户名	长度需求	6~18位	[6,18]	6	A01	空		B01
				18	A02	[1,6)	5	B02
				10	A03	>18	19	B03
	组成需求	字母、数字、下画线	字母		A04	特殊符号		B04
			字母+数字+下画线		A05	汉字		B05
	格式需求	以字母开头	以字母开头		A06	数字开头		B06
						以下画线开头		B07
		以字母或数字结尾	以字母结尾		A07	以下画线结尾		B08
			以数字结尾		A08			
密码	非空要求	不能为空	非空		A09	空		B09
确认密码	一致性要求	与密码一致	一致		A10	不一致		B10

• 3.3 基于组合及其优化的方法

• 3.3.1 决策表（判定表）- 功能测试方法中最严格的

- 决策表以等价类为基础，是功能性测试方法中最严格的。
- 目标：在特定条件下消除等价类测试的冗余。决策表的完备性保证一种完备的测试，但不能保证无冗余。
- 基本思想：基于强组合等价类测试得到有效域上完整的测试用例集合，并通过合并化简来消除用例间的冗余。
- 以强组合等价类测试为基础，遵循独立性假设。
- 决策表得到的是测试规则，而不是最终的用例。规则不能验证功能点正确，仅验证业务规则的正确性。

• 1、决策表的结构

• 概念

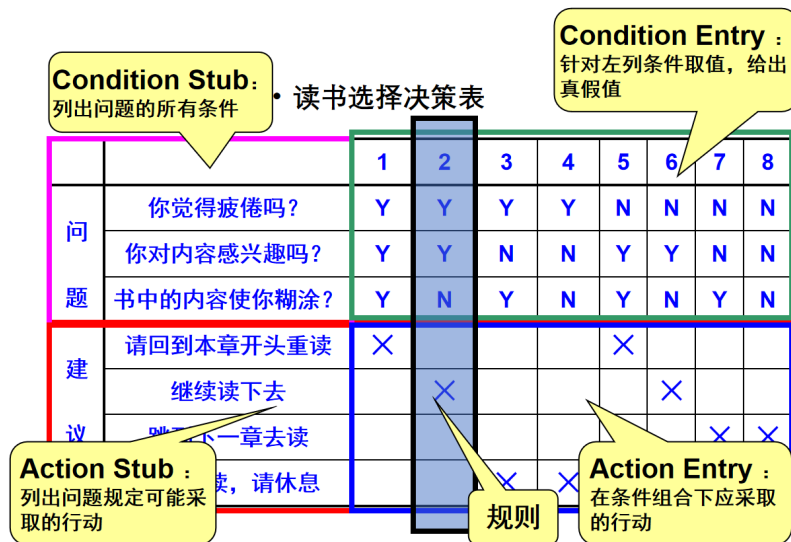
- 条件桩：问题的所有输入条件。
- 条件项：针对条件桩给出的条件，列出所有可能的取值。
- 动作桩：问题规定可能采取的操作。
- 动作项：在条件项的各组取值情况下，应采取的动作。
- 规则：任何一个条件组合的特定取值及相应要执行的动作称为一条规则。在决策表中，贯穿条件项和动作项的一列就是一条规则。可根据每个合法输入组合的规则设计测试用例。

• 格式

输入条件	条件桩	条件项1	规则	
		条件项2		
输出响应	动作桩	动作项		

• 示例

- 1、如果觉得疲倦并且对书的内容感兴趣，同时书中的内容让你糊涂的话，回到本章重读。
- 2、如果觉得疲倦并且对书的内容感兴趣，同时书中的内容不让你糊涂，继续读下去。
- 3、不觉得疲倦并且对书的内容感兴趣，但是书中的内容让你糊涂的话，回到本章重读。
- 4、如果觉得疲倦并且对书中的内容不感兴趣，同时书中的内容不让你糊涂，停止阅读，请休息。
- 5、如果觉得疲倦并且对书的内容不感兴趣，并且书中的内容让你糊涂，请停止阅读，休息。
- 6、不疲倦，对书的内容感兴趣，书中的内容不糊涂，继续读下去。
- 7、不疲倦，不感兴趣，书中内容糊涂，跳到下一章去读。
- 8、不疲倦，不感兴趣，书中内容不糊涂，跳到下一章去读。



2、决策表的化简

- 若决策表中某条件桩任意取值对动作桩无影响时, 可根据实际情况进行合并。
- 示例

- 合并前

		1	2	3	4	5	6	7	8
条件	是否疲倦	0	0	0	0	1	1	1	1
	是否感兴趣	0	0	1	1	0	0	1	1
	是否糊涂	0	1	0	1	0	1	0	1
结果	本章重读				1				1
	继续阅读			1				1	
	停止阅读					1	1		
	跳到下一章	1	1						

- 根据合并规则, { (1)、(2) }, { (3)、(7) }, { (4)、(8) }, { (5)、(6) } 可以合并。合并后:

		1、2	3、7	4、8	5、6
条件	是否疲倦	0	-	-	1
	是否感兴趣	0	1	1	0
	是否糊涂	-	0	1	-
结果	本章重读			1	
	继续阅读		1		
	停止阅读				1
	跳到下一章	1			

- 减少了规则数, 但容易遗漏分支从而产生漏测风险, 所以一般规则数量不多时尽量不合并。

- 总结

- **对测试用例进行化简的前提条件**

- **输出相同**：欲化简的多个测试用例的预期输出结果应完全相同；
- **输入相似**：欲化简的多个测试用例的输入应相似，仅有一个输入条件取值(即对应的等价类)可以不相同，其他输入条件的取值(即对应的等价类)应完全相同。
- **化简的结果**是使得某个输入条件的取值对测试用例的输出无影响。

- **3、决策表测试的步骤**

- 示例1

- **问题要求**：“对功率大于50马力的机器、维修记录不全或已运行10年以上的机器，应给予优先的维修处理.....”
- 按5步建立判定表

- **计算规则数**
- **列出所有条件、动作**
- **填入条件项和动作项**

		1	2	3	4	5	6	7	8
条件	功率大于50马力吗？	Y	Y	Y	Y	N	N	N	N
	维修记录不全吗？	Y	Y	N	N	Y	Y	N	N
	运行超过10年吗？	Y	N	Y	N	Y	N	Y	N
动作	进行优先处理	x	x	X		X		X	
	作其它处理				X		x		x



- **化简，合并规则**

		1	2	3	4	5
条件	功率大于50马力吗？	Y	Y	Y	N	N
	维修记录不全吗？	Y	N	N	-	-
	运行超过10年吗？	-	Y	N	Y	N
动作	进行优先处理	x	x		X	
	作其它处理			x		x

- 示例2

• 举例2：三角形问题

三角形类型？

输出的等价类
R1={<a,b,c> 有三条边a,b,c的等边三角形}
R2={<a,b,c> 有三条边a,b,c的等腰三角形}
R3={<a,b,c> 有三条边a,b,c的不等边三角形}
R4={<a,b,c> 三条边a,b,c不构成三角形}

能构成三角形？

• 三角形问题的决策表

C1:a,b,c构成三角形?	N	Y	Y	Y	Y	Y	Y	Y	Y
C2:a=b吗?	-	Y	Y	Y	Y	N	N	N	N
C3:a=c吗?	-	Y	Y	N	N	Y	Y	N	N
C4:b=c吗?	-	Y	N	Y	N	Y	N	Y	N
A1:非三角形	x								
A2:不等边三角形									x
A3:等腰三角形					x		x	x	
A4:等边三角形		x							
A5:不可能			x	x		x			

• 注意

- 若输入条件之间不存在相关性，则不需要使用决策表测试。
- 决策表测试法不处理无效等价类，不需要遵循单缺陷假设。
- 决策表测试法仅针对输入域展开分析，需要从设计角度对输出域进行细化。

• 练习

• 实验2附录1

1、课时津贴发放决策表

功能需求：

某校制定了教师的讲课课时津贴标准。对于各种性质的讲座，无论教师是什么职称，每课时津贴费一律是 50 元；

而对于一般的授课，则根据教师的职称来决定每课时津贴费：教授 30 元，副教授 25 元，讲师 20 元，助教 15 元。

请画出该功能的决策表。

• 实验2附录2

2、文件操作决策表

操作要求：

输入的第一个字符必须为#或*，第二个字符必须是一个数字，在此情况下进行文件修改；如果第一个字符不是#或*，则给出信息 N；如果第二个字符不是数字，则给出信息 M。

请画出该功能的决策表。提示：这两个字符都是必填的，不需要考虑不填的情况。

• 3.3.2 因果图法 - 考虑输入之间的关系

• 1、因果图的作用

- 如果程序输入之间有作用关系，等价类划分法和边界值分析法很难描述这些输入之间的作用关系，无法保证测试效果。

- 如输入地址具体到市区，如“北京—昌平区”，其中第二个输入受到第一个输入的约束，输入的地区只能在输入的城市中选择，否则地址就是无效的。
- 因果图法就是为了解决多个输入之间的作用关系而产生的测试用例设计方法，便于生成判定表。每种组合条件就是“因”，它必然有一个输出的结果，这就是“果”。
- 因果图是一种形式化的图形语言。

2、因果图的基本图形符号

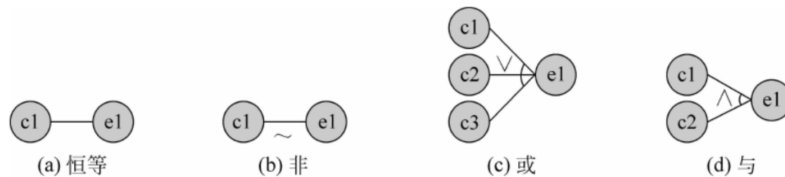
1. 输入与输出

- 输入与输出间的关系主要有恒等、非、与、或等4种。

1. 输入与输出

输入与输出间的关系主要有恒等、非、与、或等4种。

- 恒等**：要求程序有一个输入和一个输出，输出与输入保持一致。若c1为1，则e1也为1，若c1为0，则e1也为0。
- 非 (~)**：要求程序有一个输入和一个输出，输出是输入的取反。若c1为1，则e1为0，若c1为0，则e1为1。
- 或 (v)**：要求程序可以有任意个输入，只要这些输入中有一个为1，则输出为1，否则输出为0。
- 与 (^)**：要求程序也可以有任意个输入，但只有这些输入全部为1，输出才能为1，否则输出为0。

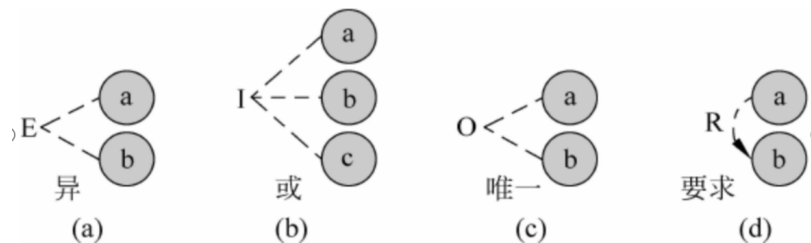


2. 输入与输入

- 输入与输入之间同样存在异、或、唯一、要求等4种关系。

多个输入之间的强约束符号如下：

- E (Exclusive, 互斥)**：表示a、b两个原因不会同时成立，两个中最多有一个成立。即a和b中最多只能有一个为1，a和b不能同时为1。**可以不选。**
- I (include, at least one, 包含)**：表示a、b、c这3个原因中至少有一个必须成立。即a、b和c中**至少有一个**必须是1，即a、b、c不能同时为0。
- O (one and only one, 惟一)**：表示a和b当中必须有一个，且仅有一个成立。即a和b有且仅有一个为1。**必须选一个。**
- R (Requires, 要求)**：表示当a出现时，b必须也出现。a出现时不可能b不出现。即a和b必须保持一致，即a=1时，b也必须为1，a=0时，b也必须为0。

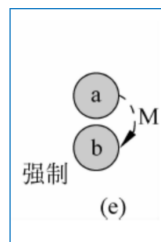


3. 输出条件的强约束关系

- **M (Mask, 强制)**：表示当a=1时，b强制为0。而当a=0时，b强制为1。

- **又称为屏蔽**

- **注**：当原因或结果成立（或出现）时，相应的节点取值为1，否则取值为0。



● 案例

● 3、利用因果图设计测试用例的步骤

1. 分析程序规格说明书描述内容，确定程序的输入与输出，即确定“原因”和“结果”，可根据输入、输出条件的等价类来确定。**原因是输入条件或输入条件的等价类，而结果是输出条件。**
2. 分析程序规格说明的描述中语义的内容，输入与输入之间、输入与输出之间的对应关系，并将其表示成连接各个原因与各个结果的“因果图”。
3. **标明约束条件。**由于语法或环境的限制，有些原因和结果的组合情况是不可能出现的。为表明这些特定的情况，在因果图上使用若干个标准的符号标明约束条件。
4. **把因果图转换成决策表。**
5. 为决策表中每一列表示的情况设计一个测试用例。

□ 因果图转换为决策表的方法：

1. 将所有原因作为输入条件项，每一项原因设为一行，列出所有条件取值（真值为1，假值为0）。
2. 将每种条件组合设为一列，并将各条件的取值情况分别添加到决策表对应的单元格中。
3. 根据条件之间的约束关系，排除无效的条件组合。

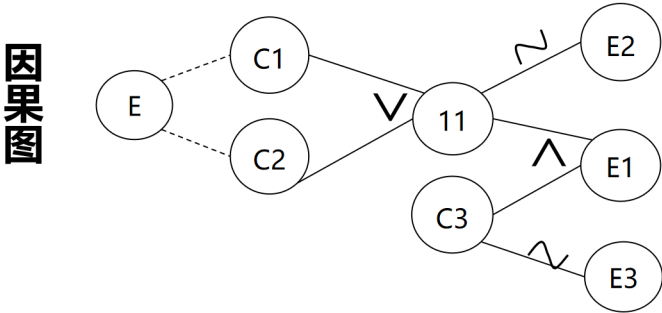
● 示例

□ 某个软件规格说明书中包含以下要求：

- 第一个字符必须是A或B，第二个字符必须是一个数字，在此情况下进行文件的修改；
- 但如果第一列字符不正确，则输出信息L；
- 如果第二列字符不是数字，则给出信息M。

□ 采用因果图方法进行分析，可根据下表获得图的各种组合，其中√表示“或”、^表示“与”、~表示“非”的关系。

编号	原因（Cause）	编号	结果（Effect）
C1	第一个字符是A	E1	修改文件
C2	第一个字符是B	E2	给出信息L
C3	第二列字符是一个数字	E3	给出信息M
11	中间原因		



转换为决策表，每一列作为确定测试用例的依据

序号		1	2	3	4	5	6
输入	C1	1	0	0	1	0	0
	C2	0	1	0	0	1	0
	C3	1	1	1	0	0	0
输出	E1	1	1	0	0	0	0
	E2	0	0	1	0	0	0
	E3	0	0	0	1	1	1

化简后的测试用例

序号		1	2	3	4/5/6
原因	C1	1	0	0	——
	C2	0	1	0	——
	C3	1	1	1	0
结果	E1	1	1	0	0
	E2	0	0	1	0
	E3	0	0	0	1
用例		首字符为A, 第2个字符为数字	首字符为B, 第2个字符为数字	首字符为x, 第2个字符为数字	首字符为A或B或x, 第2个字符不是数字

• 4、因果图法的特点与小结

• 因果图法的特点：

- 1. 考虑输入条件的组合关系。
- 2. 考虑输出条件对输入条件的信赖关系，即因果关系。
- 3. 测试用例发现错误的效率高。
- 4. 可检查出功能说明书中的某些不一致或遗漏。
- 5. 适合于检查程序输入条件和各种组合情况。

• 小结

- 因果图在实际使用过程中，能够帮助测试用例设计者快速了解需求，理解业务逻辑，然后快速设决策表，从而得到所需的测试用例。
- 在因果关系复杂的系统中，可采用该方法，该方法的优点类似于决策表法，在使用过程中需注意规则的规模。

• 3.3.3 Pairwise方法

(两两组合)

- 背景：大部分缺陷是在两个变量取值冲突的测试时被发现
- 处理的问题：多个变量、每个变量有多个取值的组合数太大（不再是条件成立、不成立两种情况）
- 定义：确保某个变量取值和另一个变量取值成对出现都会被覆盖
- 效果：可大幅度降低组合的数量

• 3.3.4 正交实验法

- 1、使用原因
 - 在许多应用系统的测试工作中，输入条件的因素很多，而且每个因素也不能简单用“是”和“否”来回答。测试组合会变得很多，如果按照传统的测试方法，会导致很大的测试工作量。
- 2、正交表的结构

因子-状态 又称因素-水平

因子	因子的状态		
操作系统	Windows	linux	Mac
数据库	MySQL	MongoDB	Oracle

列号 试验号	1	2	3	4	5	6	7	8	9	10	11
1	1	1	1	1	1	1	1	1	1	1	1
2	1	1	1	1	1	2	2	2	2	2	2
3	1	1	2	2	2	1	1	1	2	2	2
4	1	2	1	2	2	1	2	2	1	1	2
5	1	2	2	1	2	2	1	2	1	2	1
6	1	2	2	2	1	2	2	1	2	1	1
7	2	1	2	2	1	1	2	2	1	2	1
8	2	1	2	1	2	2	2	1	1	1	2
9	2	1	1	2	2	2	1	2	2	1	1
10	2	2	2	1	1	1	1	2	2	1	2
11	2	2	1	2	1	2	1	1	1	2	2
12	2	2	1	1	2	1	2	1	2	2	1

基于正交表的测试

- 根据正交性原理，从全面试验中挑选部分有代表性的试验点，并能求出最佳工艺参数和工艺条件，这些代表性试验点的特性为：
 - 整齐可比
 - 均匀分散
- 保证每个实验因子及其取值都能参与实验，减少了人为测试习惯导致覆盖率低及冗余测试用例的风险。

同一正交表中，每个因子的每个水平出现的次数完全相同。

同一正交表中，任意两列（两个因子）的水平搭配（横向形成的数字对）完全相同。

正交表的公式

$$L_n(q^s)$$

- n : 实际测试用例的个数，对应正交表的行数；
- q : 每个输入条件所取测试数据的个数，对应正交表中每个输入条件的取值个数；
- s : 输入条件的总数，对应正交表的列数；
- q^s : 理论上全组合方式的测试用例个数，基于正交表的测试效率为 n 与 q^s 的比值

3、正交表的类型

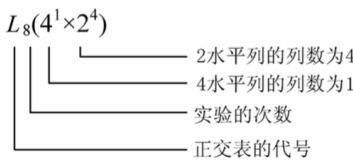
1.等水平正交表

等水平正交表即各列水平数均相同的正交表，又称单一水平正交表。



2.混合水平正交表

各列水平数不相同的正交表，又称混合水平正交表。



• 3因子2状态 $L_4(2^3)$

Experiment Number	Column 1	2	3
1	1	1	1
2	1	2	2
3	2	1	2
4	2	2	1

• 7因子2状态 $L_8(2^7)$

Experiment Number	1	2	3	4	5	6	7
1	1	1	1	1	1	1	1
2	1	1	1	2	2	2	2
3	1	2	2	1	1	2	2
4	1	2	2	2	2	1	1
5	2	1	2	1	2	1	2
6	2	1	2	2	1	2	1
7	2	2	1	1	2	2	1
8	2	2	1	2	1	1	2

2024-6-15

• 4因子3状态 $L_9(3^4)$

	1	2	3	4	y
1	1	1	1	1	y1
2	1	2	2	2	y2
3	1	3	3	3	y3
4	2	1	2	3	y4
5	2	2	3	1	y5
6	2	3	1	2	y6
7	3	1	3	2	y7
8	3	2	1	3	y8
9	3	3	2	1	y9

- 4、应用场合
 - 输入条件组合较多，且相互不影响。
- 5、性质和特点

- **正交表的性质：**

1. 每一列中每个输入条件的各个测试数据（取值）出现的次数相同。（整齐可比）
2. 任意两列所构成的各有序数对出现的次数相同。（均匀分散）

- **正交表测试法的特点：**

1. 无需穷举所有组合情况，只需少量抽样组合数据构成测试用例，明显提高效率。
2. 抽样出的组合数据分布均匀，具有最佳覆盖性。
3. 能减少手动对组合数据抽样导致的大量测试遗漏。

- 6、测试步骤

1. 确定影响功能的因子与状态
2. 选择一个合适的正交表
3. 利用正交表构造测试数据集

- **测试用例的设计**

- 将整体输入域拆分为个体输入域，确定所有输入条件及其最大取值范围；
- 确定每个输入条件的取值个数；
- 根据输入条件个数和取值个数选择合适的正交表；
- 建立正交表，并将控件和控件取值映射到正交表中，遵循独立性假设；
- 每一行是一种组合，编写一条用例；
- 测试结果分析。

- **案例**

案例1：客户查询功能

对于软件测试而言，因子即被测对象所需的测试输入条件，水平即每个输入的取值。

下图查询功能示例图所示的功能界面包含功能客户姓名、联系电话、通信地址等3个查询字段，每个查询条件有输入数据和不输入两种情况，可称之为3因子2水平。

客户姓名		联系电话		通讯地址		查询
------	----------------------	------	----------------------	------	----------------------	----

上述案例中共有3个查询字段，如果从经验测试角度来看，可测试两种情况，即3个查询字段都不输入和都输入的情况，如果从全排列角度考虑，可设计2³，即8条用例进行覆盖，但如果测试条件增加，用例数将会无比庞大，测试效率无法保证，造成测试遗漏。

如果采用正交实验方法，则可降低此类风险。

使用正交实验方法如下：

(1) 分析需求获取因子及水平

根据被测对象的需求描述，获取输入条件及每个条件可能的取值，如果取值较多，可使用等价类及边界值方法先优化。根据用户需求可以确定的因子数为3，每个因子从输入和不输入两种情况考虑，则水平为2。

(2) 根据因子及水平数选择正交表

由步骤（1）分析得知，被测对象可能所需的正交表为3因子2水平，从正交实验网站查找得知有恰好符合3因子2水平的正交表。

- 如果预估正交表与实际正交表不相符，可选择因子及水平大于预估正交表，且实验次数最少的正交表。

Experiment Number	Column		
	1	2	3
1	1	1	1
2	1	2	2
3	2	1	2
4	2	2	1

(3) 替换因子水平，获取实验次数

将输入项及取值替换正交表，获取实验次数，替换后的表格如下表所示。

实验次数	输入条件		
	客户姓名	联系电话	通信地址
1	输入	输入	输入
2	输入	不输入	不输入
3	不输入	输入	不输入
4	不输入	不输入	输入

(4) 根据经验补充实验次数

正交实验毕竟是通过数学方法推导出来的实验次数，保证了每个参与实验因子的水平取值均匀分布在实验数据中，并不能全部代表业务的实际情况，所以一般仍需要根据测试经验补充一些用例，针对上述案例，发现3因子2水平正交表并不包含**每个因子取2值**的实验，故需补充该用例，调整后的表格如右表所示。

实验次数	输入条件		
	客户姓名	联系电话	通信地址
1	输入	输入	输入
2	输入	不输入	不输入
3	不输入	输入	不输入
4	不输入	不输入	输入
5	不输入	不输入	不输入

这样，如果使用全排列测试方法得到的用例将是2³共8条用例，如果使用正交实验方法，8条用例减少至5条，同样能保证测试效果，但测试用例数量大大减少。

7、正交表和决策表的区别

- 正交表与决策表的区别：
 - 决策表法适合测试组合数量比较少的情況(一般少于 20 种)；
 - 正交表法适合测试组合数量较多的情况(一般超过 20种)；
 - 决策表法除了考虑控件之间的组合以外还要考虑控件之间的限制关系，而正交表法不需要考虑控件之间的关系。

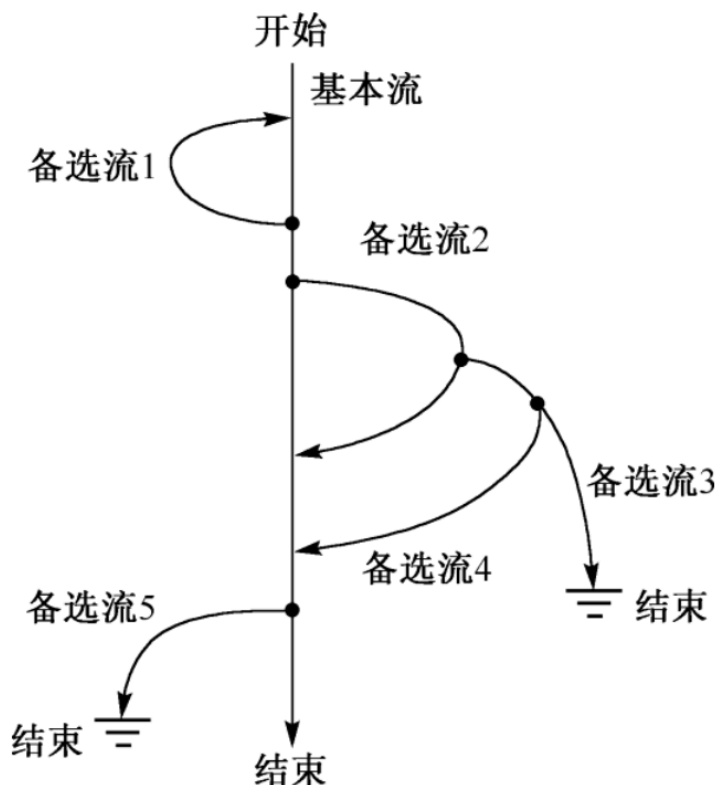
补充：场景法

概念

什么是场景？

- 现在的软件几乎都是由事件触发来控制流程的，事件触发时的情景便形成了场景，而同一事件不同的触发顺序和处理结果形成事件流。
- 场景主要描述的是系统的核心业务功能的流程。
- 场景法是一种基于软件业务的测试方法，测试人员把自己当成最终用户尽可能真实的模拟用户在使用该软件时的各种情景：
 - A. 模拟用户正确的业务操作流程——验证软件的业务功能是否能够正确实现；
 - B. 模拟用户错误的业务操作过程——验证软件的异常处理能力(健壮性)。

- 在场景的基础上进行的测试，通过执行测试场景或与系统可操作的流程相关的测试用例来验证系统的功能。
- 一个场景测试用例仅测试一个场景、事务或业务流程。
- 针对场景业务流，通常分为基本流、备选流及异常流3种业务流向。
 - **基本流**：表示输入经过每一个正确的流程运转最终达到预期结果；
 - **备选流**：表示输入经过每一个流程运转时可能产生异常情况，但经过纠正后仍能达到预期结果；
 - **异常流**：表示输入经过每一个流程运转时，产生异常终止的现象。
 - 基本流和备选流，通常作为业务流程测试过程中**优先级较高**的测试分支，应详细设计定义。异常流作为可靠性、健壮性用例亦需同步考虑。
- 基本流和备选流的区别
 - 基本流是一条从初始状态到终止状态的完整的业务流程，而备选流仅仅是业务流程中的一个执行片段，根据起始节点和终止节点的形式不同，得到的备选流的类型不同。



- 场景设计的基本原则

- 最少的场景数等于事件流的总数，即基本流与备选流的总数；
- 有且唯一有一个场景仅包含基本流；
- 对应某个备选流，至少应有一个场景覆盖该备选流，且在该场景中应尽量避免覆盖其他的备选流；
- 重复的节点不可作为新的场景，每个场景应包含从未包含的节点。

- 测试步骤

- 基本流和备选流的设置；
- 场景的构建；
- 测试用例的设计：**一个测试用例唯一对应一个场景；**
- 重新审核设计的测试用例，去掉多余的测试用例，对每个测试用例补充测试数据；
- 确定测试用例后，针对最终确定的测试用例设计测试数据并实施测试。

- 应用

- 针对单点功能利用等价类、边界值、判定表等用例设计方法能够解决大部分问题，但涉及业务流程的软件系统，采用场景法测试是比较恰当的。
- 场景法应用于冒烟测试中。
 - 冒烟测试
 - 软件构建版本建立后，对系统的基本功能进行简单的测试。
 - 这种测试重点验证的是程序的主要功能，而不会对具体功能进行深入测试。
 - 在测试时，并不是先关注某个控件的等价类，边界值这些细节，而是先关注整个功能的业务逻辑和业务流程是否正确实现，因此在开始进行测试之前，应该先做冒烟测试。

- 场景示例

场景1

登录系统
认证登录
输入主题词进行查询
获取课程列表
显示课程列表
选择课程
确认课程可选
显示最终的课程表

场景2

登录系统 认证登录
输入主题词进行查询
无效主题词
再次输入课程
获取课程列表
显示课程列表
选择课程
确认课程可选
显示最终的课程表

- 案例：微信钱包的手机充值功能
 - 使用微信钱包中的手机充值，输入手机号码，选择充值话费金额，选择支付方式，输入支付密码，支付成功。
- 1. 通过上述描述，确定基本流和备选流，如下表所示。

基本流	使用微信钱包中的手机充值，输入手机号码，选择充值话费金额，选择支付方式，输入支付密码，支付成功
备选流 1	手机号码格式输入错误
备选流 2	手机号码输入与要充值号码不符
备选流 3	银行卡账户余额不足
备选流 4	支付密码输入错误

2. 根据基本流和备选流设计场景，如下表所示。

场景 1-充值成功	基本流	
场景 2-手机号码格式输入错误	基本流	备选流 1
场景 3-手机号码输入与要充值号码不符	基本流	备选流 2
场景 4-银行卡账户余额不足	基本流	备选流 3
场景 5-支付密码输入错误	基本流	备选流 4

3. 对每个场景成立的条件都做出了说明，如下表所示，各行代表各个测试用例，而各列则代表测试用例的信息。

用例ID	场景/条件	手机号	话费金额	支付方式	支付密码	账户余额	预期结果
1	场景1：充值成功	V	V	V	V	V	充值成功
2	场景2：手机号格式输入错误	I	N/A	N/A	N/A	N/A	返回重新输入手机号步骤
3	场景3：手机号输入与要充值号码不符	I	V	V	V	V	充值成功，但充值金额未到需充值手机号
4	场景4：账户余额不足	V	V	I	N/A	N/A	提示账户余额不足，返回选择支付方式步骤
5	场景5：支付密码输入错误	V	V	V	I	N/A	提示支付密码输入错误，请重新输入

4. 设计测试用例的测试数据，如下表所示。

用例 ID	场景/条件	操作步骤/测试数据	预期结果
1	场景 1：充值成功	1. 手机号码输入正确 2. 密码输入正确 3. 账户余额充足	充值成功
2	场景 2：手机号码格式输入错误	1. 手机号码格式输入 123456789012	返回基本步骤 1 重新输入手机号
3	场景 3：手机号码输入与要充值号码不符	1. 手机号码输入 13720008015（应充值号码为：13720008014）	充值成功，但充值金额未到需充值手机号
4	场景 4：银行卡账户余额不足	1. 手机号码输入正确 2. 密码输入正确 3. 账户余额不足	提示账户余额不足，返回选择账户步骤
5	场景 5：支付密码输入错误	1. 手机号码输入正确 2. 密码输入错误	提示密码输入错误，请重新输入

- 场景法小结
 - 通过分析被测业务流程，构建基本流和备选流，并生成场景进而得到测试用例的测试方法。该法主要用于**功能测试**。
 - 主要**难点**在于如何根据业务实际提炼出基本流，如何很好地控制备选流的数量，以及如何从无穷尽的场景中选择少量典型场景进行测试。

● 白盒测试

白盒测试(White-box Testing)

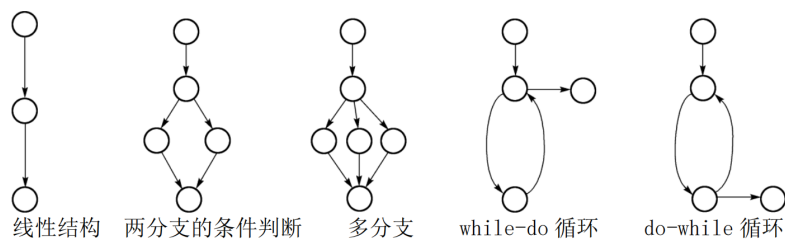
白盒测试又称为结构测试或逻辑测试，它一般用来分析程序的内部结构，针对程序的逻辑结构来设计测试用例进行测试。

白盒测试的测试目的是，通过检查软件内部的逻辑结构，对软件中的逻辑路径进行覆盖测试；在程序不同地方设立检查点，检查程序的状态，以确定实际运行状态与预期状态是否一致。

主要包含六种测试方法：**语句覆盖、判定覆盖、条件覆盖、判定条件覆盖、条件组合覆盖、路径覆盖**。

● 3.4 基于逻辑覆盖的方法

● 控制流图



● 1、语句覆盖

- 基本思想：设计测试用例时要保证程序的每一条语句至少执行一次。

案例：

```
if A and B then Action1
if C or D then Action2
```

上例中，设计一条测试用例即可：A=true B=true C=true

- 语句覆盖等同于对流程图中所有节点的覆盖。
- 局限性：语句覆盖是“最弱的覆盖”，只考虑了语句而非判定表达式。
- 2、判定覆盖（分支覆盖）
 - 基本思想：使程序中每个判断的取真和取假至少经历一次，即判断的真假均曾被满足。

案例：

```
if A and B then Action1
if C or D then Action2
```

- 先找出所有的判定：A and B 和 C or D。然后让所有的判定都分别取真一次，取假一次。

A and B == True

A and B == False

C or D == True

C or D == False

上例中，**两条测试用例**即可：

testcase1: A=true, B=true, C=true, D=false

testcase2: A=true, B=false, C=false, D=false

- 判定覆盖等同于对流程图中所有边的覆盖。
- 局限性：未彻底分析每个简单判定条件的取值情况，仍然会导致遗漏部分缺陷。
- 3、条件覆盖
 - 基本思想：条件覆盖要求所设计的测试用例能使**每个判定中的每一个条件都获得可能的取值**，即每个条件至少取一次真值、取一次假值。只考虑条件本身，不考虑整个判定节点的真假情况。

案例：

```
if A and B then Action1
if C or D then Action2
```

第一个判断应考虑到A=true, A=false, B=true, B=false
第二个判断应考虑到C = true, C = false, D=true, D=false,

所以上例中可以设计测试用例满足下列条件，**两条用例：**

testcase1: A=true, B=true, C = true, D=true

testcase2: A=false, B=false, C = false, D=false

- 局限性：条件覆盖不能保证100%的判定覆盖。

• 4、判定-条件覆盖

- 基本思想：测试用例可以使得判定中每个条件所有的可能取值都至少执行一次（条件覆盖），同时每个判定本身所有的可能取值也至少执行一次（判定覆盖）。

案例：

```
if A and B then Action1
if C or D then Action2
```

条件覆盖：A=true, A=false, B=true, B=false
C = true, C = false, D=true, D=false,

判定覆盖：A and B =True , A and B =False
C or D =True , C or D =False

所以满足条件覆盖和判定覆盖，我们可以设计**两条用例：**

TestCase1: A=true, B=true, C = true, D=true （条件都为True, A and B=True, C or D =True）

Testcase2: A=false, B=false, C = false, D=false （条件都为False, A and B=False, C or D =False）

- 局限性：判定条件覆盖并未考虑条件的组合情况。

• 修正的判定-条件覆盖：

- 修正方式

即**修正条件判定覆盖**首先要求实现条件覆盖、判定覆盖，在此基础上，对于每一个条件C，要求存在符合以下条件的两次计算：

- 1) 条件C所在判定内的所有条件，除条件C外，其他条件的取值完全相同；
- 2) 条件C的取值相反；
- 3) 判定的计算结果相反。

• 对于表达式：A AND B

- 体现A对判定结果的独立影响性： T1, T3
- 体现B对判定结果的独立影响性： T1, T2
- 最终用例集合： T1~T3

	T1	T2	T3	T4
A	T	T	F	F
B	T	F	T	F
A and B	T	F	F	F

- 最终抽取最少独立影响对，即为测试用例集。

• 5、组合覆盖（条件组合覆盖）

- 基本思想：测试用例应该使得每个判定中的各个条件的各种可能的组合都至少出现一次。实质是通过列出真值表的方式来得到完全的覆盖，即以冗余换取方法的简单性。

满足条件组合覆盖的测试用例一定是满足判定覆盖、条件覆盖和判定-条件覆盖的。

案例：

```
if A and B then Action1
if C or D then Action2
```

TestCase1: A=True, B=True, C=True,D=True

TestCase2: A=True, B=False, C=True,D=False

TestCase3: A=False, B=True, C=False,D=True

TestCase4: A=False, B=False, C=False,D=False

- A和B的每一种组合情况都要覆盖到，同理，C和D的每一种组合情况也都要覆盖到。
 - 覆盖率较高，但测试用例数量也较多。
- 练习（实验4-1）

1、 分析以下程序代码，其中 P1、P2 为判定条件，请设计测试用例使其满足语句覆盖、判定覆盖、条件覆盖、判定条件覆盖、条件组合覆盖。

```
1  int DoWork(int x, int y, int z)
2  {
3      int k=0, j=35;
4      if((x<3)&&(z<10))    //P1
5      {
6          k=x*y-1;
7          j=sqrt(k);
8      }
9      else if((x==3) || (y>5))    //P2
10     {
11         j=x*y+10;
12     }
13     else
14         j=j%6;
15     return j;
16 }
```

- **6、基本路径覆盖**

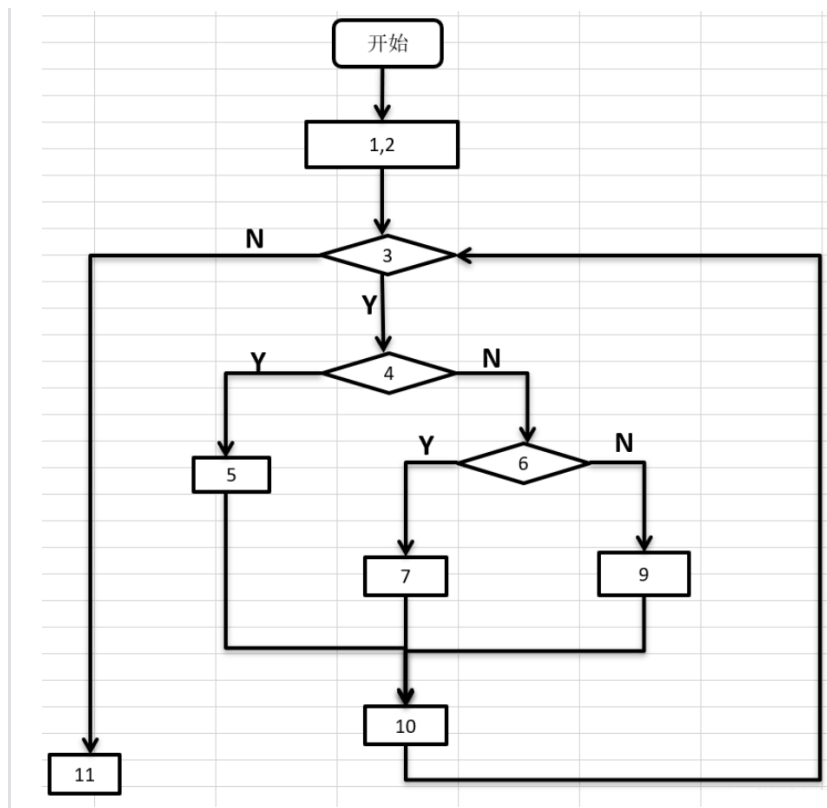
- 示例·

```

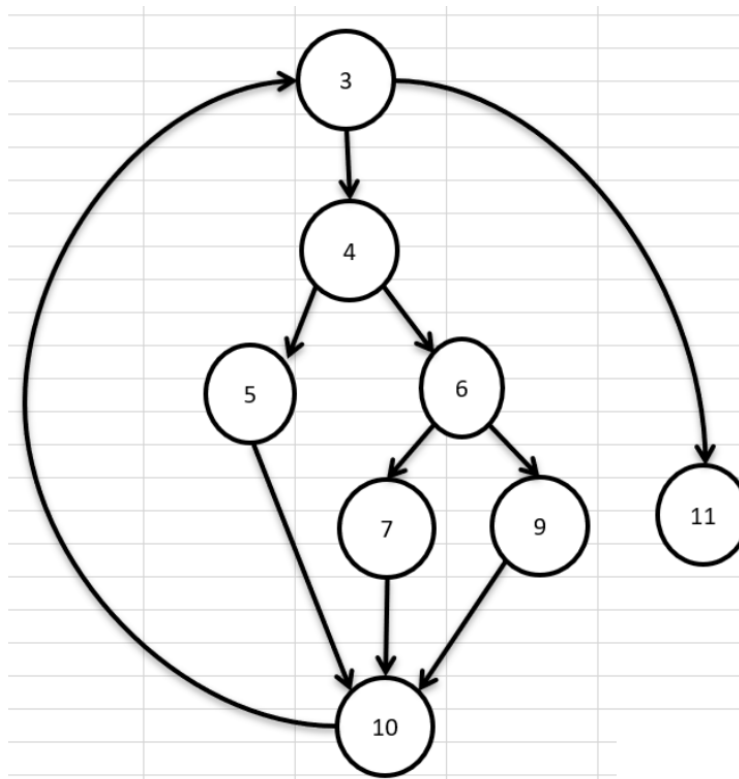
void Sort ( int iRecordNum, int iType ) {
1      int x=0;
2      int y=0;
3      while ( iRecordNum-- > 0 ) {
4          if ( iType==0 )
5              x=y+2;
6          else if ( iType==1 )
7              x=y+10;
8          else
9              x=y+20;
10     }
11 }

```

- (1) 根据代码画出程序流程图



- (2) 画出控制流图

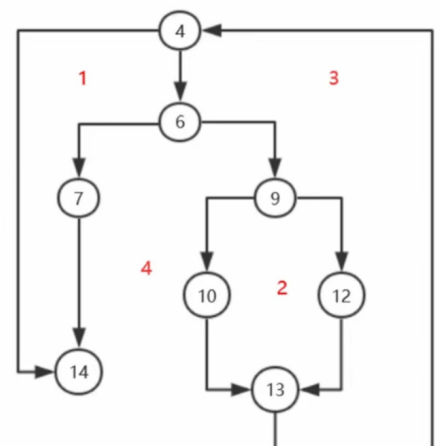


• 程序图压缩的原则

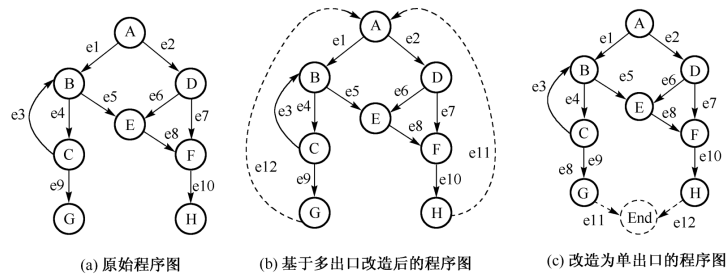
- 剔除注释语句
 - 剔除数据变量的声明语句
 - 所有连续的串行语句压缩为一个
 - 所有循环次数压缩为一次循环
- (3) 根据程序环复杂度的计算公式，求出程序路径集合中的独立路径数目
 - 计算方式（下图与上面的示例无关）

方法有三种：

- ◆ 流图中区域的数量对应于环型的复杂性。
- ◆ 给定流图G的圈复杂度 $V(G)$ ，定义为
 $V(G) = E - N + 2$ ，E是流图中边的数量，N是流图中节点的数量。右图： $V(G) = 10 - 8 + 2$
- ◆ 给定流图G的圈复杂度 $V(G)$ ，定义为
 $V(G) = P + 1$ ，P是流图G中判定节点的数量。
 右图： $V(G) = 3 + 1$



• 环复杂度计算——多出口程序的处理



环复杂度为5

- 推荐使用 $V(G) = P + 1$ 的方式来计算环复杂度。
- 示例中， $V(G) = P + 1 = 3 + 1$ ，控制流程图环形复杂度为 4。
- (4) 根据环复杂度的计算结果，源程序的基本路径集中有多少条独立路径

path-1: 3 - 11
path-2: 3 - 4 - 5 - 10 - 3 - 11
path-3: 3 - 4 - 6 - 7 - 10 - 3 - 11
path-4: 3 - 4 - 6 - 9 - 10 - 3 - 11

- 在一个基本路径集合里，每条路径是唯一的。但基本路径集合不是唯一的。
- (5) 写出对应的测试用例

测试用例					
编号	输入数据		覆盖路径	预期结果	
	iRecordNum	iType		x	y
test-1	0	0	path-1	0	0
test-2	1	0	path-2	2	0
test-3	1	1	path-3	10	0
test-4	1	2	path-4	20	0

- 练习（实验4-2）
 - 2、 分析以下程序代码，使用白盒测试技术设计测试用例，并回答下列问题。
- ```
1 void sort(int i, int j)
2 {
3 int m,n,k;
4 m=n=k=0;
5 while(i++<11)
6 {
7 if (j<5)
8 m++;break;
9 else if (j<8)
10 n++;
11 else
12 k++;
13 }
14 printf("%d,%d,%d" , m, n, k);
15 }
```

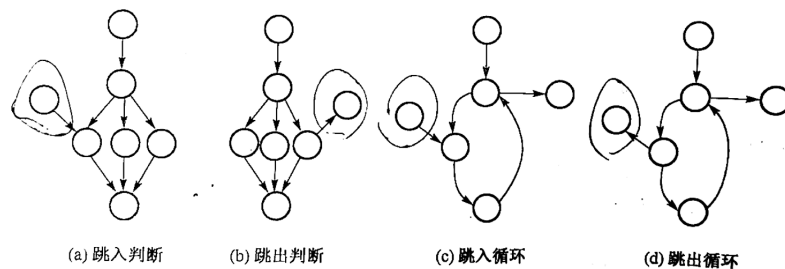
## • 小结

- 是最重要的白盒测试方法之一，其思想可用于任何动态模型中：
  - 单元测试阶段，主要用于对程序源代码的执行测试。
  - 集成测试或系统测试阶段，主要用于对业务流程、页面跳转等类似动态执行路径的测试。

## • 补充：基本复杂度

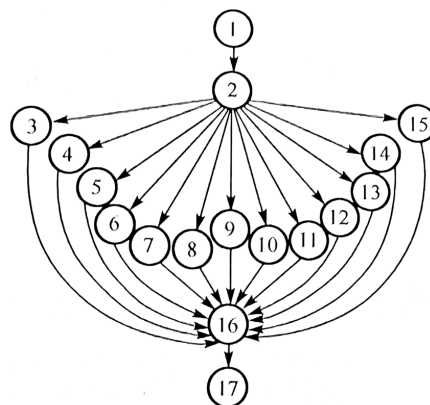
- 概念：通过对程序图中的结构化设计节点进行不断压缩，最终得到一个无法压缩的程序图，该图的环境复杂度就称为基本复杂度。
- 基本复杂度关注的是程序中所有非结构化设计的代码，包含测试优化和设计优化的思想。
- 即使程序环境复杂度较高，但若基本复杂度不高，则说明该程序多为结构化的设计，设计本身较优，引入缺陷的风险更低，也更利于分别针对被压缩的结构化设计展开独立测试。

## 非结构化程序结构



## 结构化程序结构

可将所有分支压缩为一个节点，则基本复杂度为1。



## • 基本路径测试法的目标：

- 测试的完备性：通过对独立路径的测试达到对所有路径的覆盖。
- 测试的无冗余性：每条路径都是独立的，针对每条路径设计的测试用例之间不存在冗余。

## • 独立路径的抽取原则：

- 任意两条路径线性无关（每一条路径具有不同特征）；
- 所有独立路径的并集是整个向量空间。

## • 独立路径集规模确定

- 按照McCabe的环复杂度概念，对于指定的程序图，对路径的测试中所需独立路径集合的大小就**等于其程序图的环复杂度**。

## • 独立路径抽取（难点）

### • 1、确定主路径：

- 该路径应包含尽可能多的判定节点（包括条件和循环判定）
- 应包含尽可能复杂的判定表达式（与、或关系多）
- 应对应尽可能高的执行概率
- 应包含尽可能多的语句

### • 2、根据基础路径抽取其他独立路径

- 依次在该路径上的每个判定节点从执行一个新的分支，直达到数目。

### • 举例：独立路径抽取

- Path1: A, B, C, G(经过判定节点A、B、C);
- Path2: A, D, E, F, G(在判定节点A处执行e2分支);
- Path3: A, B, E, F, G(在判定节点B处执行e5分支);
- Path4: A, B, C, B, C, G(在判定节点C处执行e3分支);
- Path5: A, D, F, G(在判定节点D处执行e7分支)

